Universidad de La Laguna

Introduction to Design Patterns

Academic Year 2025–2026

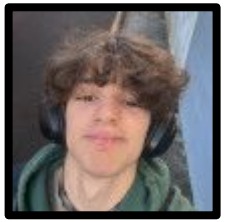
Introduction to Design Patterns

Link to code examples in these slides
Link to all code examples in this course

Don't forget to update the repos!

Contact us!

Kyrylo Chvanov (kyrylo.chvanov.20@ull.edu.es)



Adriel Reyes Suárez (adriel.reyes.suarez.33@ull.edu.es)

F. Gabriel Ruiz Ruiz (gabriel.ruiz.40@ull.edu.es)

Index

- Why should you be interested?
- What is a design pattern?
- Some background information
- The importance of design patterns
- How to use design patterns
- Classification of design patterns
- Builder pattern
- Facade pattern

Index

- Singleton pattern
- Adapter pattern
- Mediator pattern
- References

Why should you be interested?

Why should you be interested?

PAI 2526 / Práctica ?? - ??? / 2025-2026 Cuestionario ??. Introduction to Design Patterns

2025-2026 Cuestionario ??. Introduction to Design Patterns

Por hacer: Recibir una calificación

Abre: ???

Cierra: ???

Why should you be interested?

PAI 2526 / Práctica ?? - ??? / 2025-2026 Cuestionario ??. Introduction to Design Patterns

2025-2026 Cuestionario ??. Introduction to Design Patterns

Por hacer: Recibir una calificación

Abre: ???

Cierra: ???

That's a good reason, but there are better ones...

What is a design pattern?

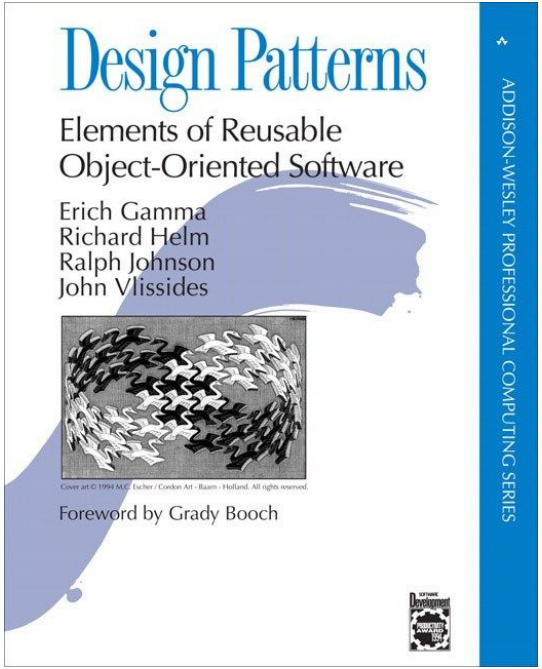
“Design patterns are typical solutions to common problems in software design.”

Source: Refactoring Guru

Some background information

Design Patterns

Gang of Four (GOF)



[More info here!]

Why are they important?

- Easy to engage in a new project

Why are they important?

- Easy to engage in a new project
- Universal understanding

Why are they important?

- Easy to engage in a new project
- Universal understanding
- Documented methodology

Why are they important?

- Easy to engage in a new project
- Universal understanding
- Documented methodology
- Industrial standards

Why are they important?

- Easy to engage in a new project
- Universal understanding
- Documented methodology
- Industrial standards

More in detail

How to use them

It is NOT about copying and pasting…

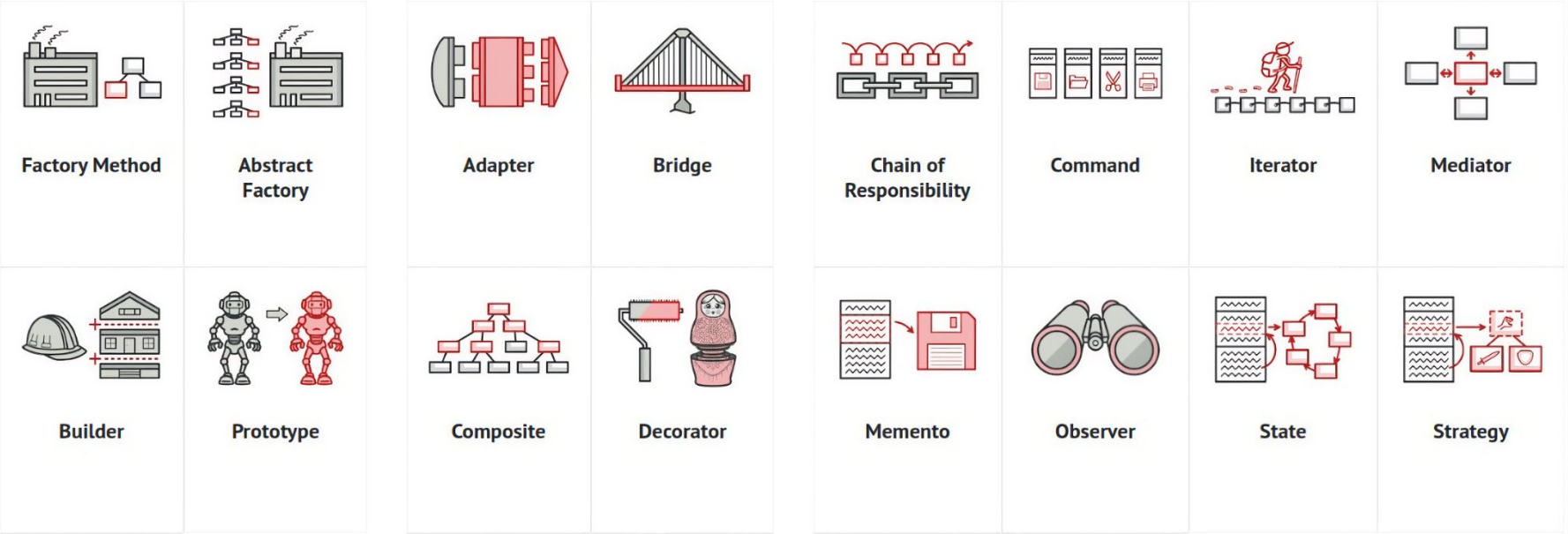
How to use them

It is NOT about copying and pasting…

…rather **computational thinking!**

There are a lot of them…

There are a lot of them…



There are a lot of them…

We will only cover the easiest to understand patterns!

Classification

Classification

- Creational

Classification

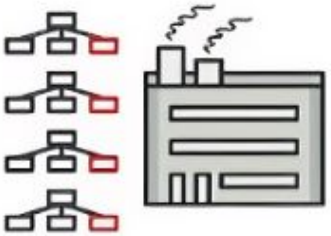
- Creational
- Structural

Classification

- Creational
- Structural
- Behavioural

Creational

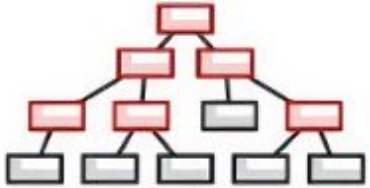
Defines how objects are created.



More about creational patterns…

Structural

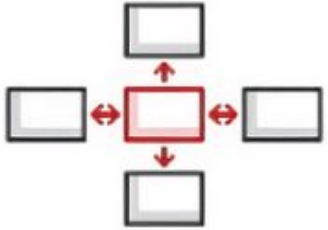
Defines how objects relate to each other.



More about structural patterns…

Behavioral

Define how objects communicate with each other.



More about behavioural patterns...

Let’s dive into a few of them…

Let’s dive into a few of them…

Objective: **introduce you to design patterns!**

Builder pattern

Intent

- Enables you to build complex objects step by step.

Intent

- Enables you to build complex objects step by step.
- Same code, different representations of an object!

Problem

Complex objects require large constructors...

Initializing them can be a hassle!

Read up on Telescoping Constructors!

Problem

```
class User {
  constructor(private name: string, private dni?: number,
      private address?: string, private: city?: string) {
    // ...
  }
  // ...
}
```

Problem

```
// I know the name and dni - constructor seems fine.
const jorge = new User(‘Jorge’, 10000);


// I only know the name and city...
const abel = new User(‘Abel’, 0, ‘’, ‘Sta. Cruz’);
// This is not comfortable to work with!
```

builder-pattern-user-problem.ts

Solution

Delegate the object construction to another object: **builders**!

Code example

We are aiming to achieve this:

```
const userBuilder = new UserBuilder(); // the builder!

const abel = userBuilder.setName('Abel').setCity('Sta.
Cruz').build();

// No need to initialize unknown variables!
```

builder-pattern-user-solution.ts

Building standardization

What if we wanted to create a standard for creating default users…

Building standardization

What if we wanted to create a standard for creating default users…

…with different builders?

Building standardization

```
const redUserBuilder = new redUserBuilder();
const red1 = redUserBuilder.setName('x').setCity('LL').build();
const blueUserBuilder = new blueUserBuilder();
const blue1 = blueUserBuilder.setName('x').setCity('LL').build();
// ...
```

DRY - Don’t Repeat Yourself!

Building standardization

Let's create an object that follows the same recipe for any* given builder: **the director!**

**they must be related somehow (interface or abstract class).*

Building standardization

```
class UserDirector {
  buildDefaultUser(builder: AbstractBuilder) {
    return builder.setName('x').setCity('LL').build();
  }
}
```

builder-pattern-user-director.ts

Building standardization

We are aiming to achieve this:

```
const director = new UserDirector();
const redUserBuilder = new redUserBuilder();
const red1 = director.buildDefaultUser(redUserBuilder);
const blueUserBuilder = new blueUserBuilder();
const blue1 = director.buildDefaultUser(blueUserBuilder);
// Director works for two different builders!
```

Building standardization

Strongly recommended read:

Why should you need directors?

Facade pattern

Intent

...

image of guru

Problem

...

image of guru

Solution

...

image guru

Real life example

...

image

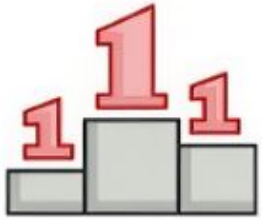
Code example

...

image

source of code

Singleton pattern



Intent

...

image of guru

Problem

...

image of guru

Solution

...

image guru

Real life example

...

image

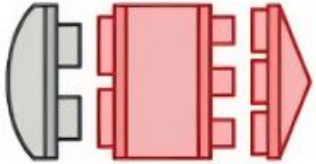
Code example

...

image

source of code

Adapter pattern



Intent

Enables objects with incompatible interfaces to collaborate.

Problem

We have designed a product for our program:

```
// Target: interface our program understands
class Product {
  constructor(protected readonly price: number) { }
  // ...
}
```

Problem

With this getter:

```
// ...
getPriceEuros(): number {
  return this.price;
}
//...
```

Problem

We have created a function that uses it:

```
function printPrice(product: Product) {
  console.log(`Price: ${product.getPriceEuros()} €`);
}
```

Problem

We now need to use an external library, but it uses Dollars instead of Euros!

Problem

```
// Adaptee: the library we need to use but is incompatible!
class ExternalService {
  getAmountInDollars(): number {
    return 10.50; // $10.50
  }
}
```

Problem

Our printer does NOT understand it...

```
const service = new ExternalService();
printPrice(service);
// Error: service isn't a Product and we can't change this!
```

adapter-pattern-money-problem.ts

Solution

We need a class that:

Solution

We need a class that:

- Implements the target (Product).

Solution

We need a class that:

- Implements the target (Product).
- Wraps the external incompatible library (ExternalService).

Solution

We need a class that:

- Implements the target (Product).
- Wraps the external incompatible library (ExternalService).
- Translates incompatible data.

Solution

These should be the changes needed:

```
const service = new ExternalService();
const adapter = new MoneyAdapter(service);
printPrice(adapter);
// Now this works!
```

Solution

This method call works wonders:

```
function printPrice(product: Product) {
  console.log(`Price: ${product.getPriceEuros() } €`);
}
```

Code example

Adapter (MoneyAdapter) inherits from the target class (Product).

```
class MoneyAdapter extends Product {
  // ...
}
```

adapter-pattern-money-solution.ts

Code example

Creates an instance using the adaptee (ExternalService):

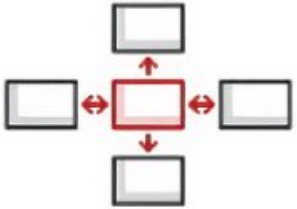
```
//...
constructor(private externalService: ExternalService) {
  super(0); // This value will be ignored later on...
}
//...
```

Code example

Overrides the target's method:

```
//...
getPriceEuros(): number {
  const dollars = this.externalService.getAmountInUSD();
  return dollars * 0.92; // Conversion rate USD -> Euros
}
//...
```

Mediator pattern



Intent

...

image of guru

Problem

...

image of guru

Solution

...

image guru

Real life example

...

image

Code example

...

image

source of code

Generic title

Generic text

Generic title

Generic text

- Generic bullet point list
- …

Generic title

- Generic bullet point list
- …

Universidad de La Laguna

Which pattern is the best?

Universidad de La Laguna

The one that best solves your problem!

Universidad de La Laguna

Any questions?

References

- C# Design Patterns (dofactory)
- Refactoring Guru
- GeeksForGeeks
- Design Patterns by the GoF (Amazon)
- Wikipedia (for DRY)
- Captain Debug's Blog